



No. Python Oops Interview Questions

Object-Oriented Programming (OOPS)?

What is `_init_` Method?

What is the use of `self` in Python?

What is `break`, `continue` and `pass` in Python?

What is `issubclass()`?

What is `_str_` and `_repr_`?

What is Concrete Method?

Difference between method and function?

What is Class?

Python get Class Variables/attributes

Set/Change values for class variables

Delete class variables

Built-In Class Functions (`getattr`, `setattr`, `delattr`, `hasattr`) In Python?

What is Object?

Delete the Object?

Counting the Number of objects of a Class?

What is Inheritance?

Single Inheritance

Multiple Inheritance

Multi-Level Inheritance

Hierarchical Inheritance

Hybrid Inheritance

What is Encapsulation?

No. Python Oops Interview Questions

What is Polymorphism?

What is a constructor

Types of Constructors (Default, Non-parametrized, Parameterized)

Constructor With Default Values?

Constructor Overloading

What is Destructors?

Python Access Modifiers?

private attributes and method?

What is the super() Function

MRO(Method Resolution Order) / Diamond Problem?

What is Abstract Class?

When use abstratc class?

How to Create an Abstract Method

Built-In Class Attributes?

What is Method Overloading?

What is Method Overriding?

What is Method?

What is static method?

Oops

Object-Oriented Programming (OOPS)

- Object-oriented programming (OOP) is a programming style that organizes code around objects, rather than functions and logic.
- Main Concepts of Object-Oriented Programming (OOPs)
- [Class](#)
- [Objects](#)
- Polymorphism
- Encapsulation
- Inheritance
- Data Abstraction

Ques. What is the use of self in Python?

- The Self parameter is a **reference** to the **current instance of the class**, we can access the attributes and methods of the class in python.
- We can give **any name** in place of self but **first parameter** is **compulsory**.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def myfunc(self):
        print("Hello my name is " + self.name)
p1 = Person("Mohit", 36)
p1.myfunc()      # Output:- Hello my name is Mohit
```

```
class Person:
    def __init__(mysillyobject, name, age):
        mysillyobject.name = name
    def myfunc(abc):
        print("Hello my name is " + abc.name)
p1 = Person("mohit")
p1.myfunc() # Output:- Hello my name is mohit
```

Ques. What is init Method?

- **init** is a constructor method in Python, and is **automatically called** to allocate memory when a **new object/instance is created**.
- All classes have a function called **init()** function, whatever you create them or not, which is always executed when the object is being initiated.

```
class Student:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname
        print("My name is " + self.firstname + " " + self.lastname)

    def stdInfo(self):
        print("My name is " + self.firstname + " " + self.lastname)

# creating a new object
stu1 = Student("Mohit", "Saxena") #output:- My name is Mohit Saxena
stu1.stdInfo() #output:- My name is Mohit Saxena

print(stu1.firstname) # Output:- Mohit
print(stu1.lastname)  # Output:- Saxena
```

Ques. What is break, continue and pass in Python?

Pass Keyword

- The pass keyword represents a **null operation** in Python.
- Without the pass statement in the following code, we may run into some errors during code execution.
- As explained above, the pass keyword in Python is generally used to fill up empty blocks and is similar to an empty statement represented by a semi-colon in languages such as Java, C++, Javascript, etc.

```
def myEmptyFunc():  
    # do nothing  
    pass  
myEmptyFunc()    # nothing happens  
## Without the pass keyword  
# File "<stdin>", line 3  
# IndentationError: expected an indented block
```

Break Keyword

- The break keyword in Python is used to exit a loop immediately, no matter where you are in the loop or how many iterations are left.

```
for i in range(5):  
    if i == 3:  
        break # Exit the loop when i is 3  
    print(i)  
  
# Output:-  
0 1 2
```

Continue Keyword

- The continue keyword in Python is used inside loops (for or while) to **skip** the rest of the **current iteration and** immediately **proceed** to the **next iteration** of the loop.

```
for i in range(5):  
    if i == 2:  
        continue # Skip the rest of the loop when i is 2  
    print(i)  
  
# Output:-  
0 1 3 4
```

Ques. What is `issubclass()`?

- This function returns **True** if the given class is the subclass of the specified class. **Otherwise**, it returns **False**.

Syntax:- `issubclass(class, classinfo)`

```
class Animal:  
    pass
```

```
class Dog(Animal):  
    pass
```

```
class Car:  
    pass
```

```
print(issubclass(Animal, Animal))    # True  
print(issubclass(Dog, Dog))          # True  
print(issubclass(Dog, Animal))       # True  
print(issubclass(Animal, Dog))       # False  
print(issubclass(Dog, (Animal, Car))) # True – Dog is a subclass of one of the  
tuple elements
```

Ques. What is str and repr?

☒ str Function

- The **str** method also known as a **dunder method/double underscore method**, that defines the string representation of an object.
- It's used to return a **human-readable** string when the built-in functions `str()` or `print()` are called on an instance of a class.

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f'I am a {self.name}'

obj = Student("mohit")
print(obj)  # Output:- I am a mohit

# Without str
# Without a __str__ method, Python uses the default representation
class Student:
    def __init__(self, name):
        self.name = name

obj = Student("mohit")
print(obj)  # Output:- <__main__.Student object at 0x7dc248f35c70>
```

☒ repr Function

- The **repr() method** returns a string containing a printable representation of an **object**.
- **repr** method returns a string representation of an object that is **machine-readable**.

```
import datetime
today = datetime.datetime.now()
print(str(today))  # 2025-05-11 10:47:08.923663 (Readable end user format)
print(repr(today)) # datetime.datetime(2025, 5, 11, 10, 47, 8, 923663) (official
developmrnt format)
```

Ques. What is Concrete Method?

- A concrete method is a method whose action is defined in the abstract class itself.

```
from abc import ABC, abstractmethod
class Father(ABC):
    @abstractmethod
    def disp(self):
        pass                                # method without body
    def show(self):
        print("concrete method")          # concrete Method / method with body
```

```
class Father:
    def __init__(self, fname, lname):
        self.first_name = fname
        self.last_name = lname

    def show_data(self):
        print(self.first_name)

obj = Father('mohit', 'saxena')
obj.show_data()
```


Ques. Difference between method and function?

Function

- A function is a block of code that performs a specific task and can be called independently from anywhere in your program.
- It is defined using the **def** keyword.
- It can take arguments (input values) and return values (output values).

```
def add_numbers(x, y):  
    return x + y  
  
result = add_numbers(5, 3)  
print(result)
```

Method

- A method is a function that is associated with an object or a class.
- It is defined within a class and operates on the data or attributes of that class.
- It is called using the dot notation on an object of the class.

```
class Dog:  
    def bark(self):  
        print("Woof!")  
  
my_dog = Dog()  
my_dog.bark()
```

What is Python Enumeration?

- In Python, an enumeration (or "enum") is a class that defines a set of symbolic names (constants) that are bound to unique, constant values. Enums are created using the enum module and are typically used to represent groups of related constants, making code more readable and maintainable.

```
from enum import Enum

class TrafficLight(Enum):
    RED = 1
    YELLOW = 2
    GREEN = 3

# Accessing enum members by name:
print(TrafficLight.RED.name) # Output: RED
print(TrafficLight.GREEN.value) # Output: 3

# Accessing enum members by value:
print(TrafficLight(1).name) # Output: RED
print(TrafficLight(3).value) # Output: 3
```

class

Ques. What is Class?

- Class is a template/blueprint/prototype for creating objects.
- class is a collection of objects/attributes and method.
- It is a logical entity that has some specific attributes and methods.
- **Example:-** Email is a class and headding, participant, attachment is object

Define a class

- We use the class keyword followed by the class name and a colon. The following example defines a Person class:

```
class Person:  
    pass
```

- If the class name contains multiple words, you use the **CamelCase** format, for **example:- SalesEmployee**.
- When printing out the person object, you'll see its name and memory address:

```
class Person:  
    pass
```

```
print(person)
```

```
Output:- <__main__.Person object at 0x000001C46D1C47F0>
```

- To get an identity of an object, you use the id() function. For example:

```
print(id(person)) # 1943155787760
```

Python get Class Variables/attributes

- Get the values of **class variables**

```
# -----using class name-----
class Student:
    name = 'mohit saxena'
    roll_no = '12845678'

print(Student.name)      # mohit saxena
print(Student.roll_no)   # 12845678

# -----using **getattr()** function:- The getattr() function accepts an object
# and a variable name. It returns the value of the class variable.
name = getattr(Student, 'name')
rollno = getattr(Student, 'roll_no')

print(name)              # mohit saxena
print(rollno)            # 12845678

# -----using object:- Accessing through object instantiation.
obj= Student()
print(obj.name) # mohit saxena
```

- If you access a class variable that **doesn't exist**, you'll get an **AttributeError** exception.

Set/Change values for class variables

- To set a value for a class variable, you use the dot notation:

```
# set the value using class
class Student:
    name = 'mohit saxena'
    roll_no = 12845678

# Changing value using Class Name
Student.roll_no = 10
print(Student.roll_no)    # output:- 10

# using setattr() built-in function
setattr(Student, 'roll_no', 10)
print(Student.roll_no)    # output:- 10

# using object
obj= Student()
obj.name = 'saxena mohit'
print(obj.name) # Output:- saxena mohit
```

Delete class variables

```
class Student:
    name = 'mohit saxena'
    roll_no = '12845678'

print(Student.name)      # mohit saxena
print(Student.roll_no)   # 12845678

# using **delattr()** function:
delattr(Student, 'roll_no') # Output:- AttributeError: type object 'Student' has
no attribute 'roll_no'

# using del keyword
del Student.roll_no
print(Student.roll_no)   # Output:- AttributeError: type object 'Student' has no
attribute 'roll_no'
```

Built-In Class Functions (getattr, setattr, delattr, hasattr) In Python?

1. getattr() Function
2. setattr() function
3. delattr() function
4. hasattr() function

getattr()

- **The getattr()** function returns the value of the specified attribute from the specified object.
- **syntex:-** getattr(object, attribute)
- **Example 1:-**

```
class Person:
    name = "John"
    age = 36
    country = "Norway"

x = getattr(Person, 'age')
print(x)      # 36

# Using Object
obj = Person()
y = getattr(obj, 'age')
print(y)      # my age is 36
print(f"my age is {y}")    # my age is 36
```

- **Example 2:-**

```
class Calculator:
    def add(self, a, b):
        return a + b

calc = Calculator()

# Accessing a method dynamically
operation = getattr(calc, "add")
result = operation(3, 5)
print(result) # Output:- 8
```

- **Example 3:-**

```
class Library:
    def __init__(self, id, name):
        self.bookId = id
        self.bookName = name
```

```
book = Library(101,"The Witches")
book.getBookName()           # The name of book is The Witches
print(getattr(book, 'bookName')) # The Witches
print(book.__dict__)
```

- when the attribute does not exist then use the "default" parameter to write a message.
- Syntax:- `getattr(object, attribute, 'kuch bhi msg')`

```
class Person:
    name = "John"
    age = 36
    country = "Norway"

x = getattr(Person, 'coun', 'varibal does not exist')
print(x) # Output:- varibal does not exist
```

setattr()

- **setattr()** function sets the value of the specified attribute of the specified object.
- If the attribute is not found the value will not be set.
- **syntax:-** `setattr(object, attribute, value)`

```
class Person:
    name = "John"
    age = 36
    country = "Norway"

setattr(Person, 'age', 40)
# The age property will now have the value: 40
x = getattr(Person, 'age')

print(x)           # 40
```

```
class Library:
    def __init__(self, id, name):
        self.bookId = id
        self.bookName = name

book = Library(101,"The Witches")
print(getattr(book, 'bookName')) # The Witches
setattr(book, 'bookName', 'python book')
print(getattr(book, 'bookName')) # python book
```

delattr()

- The **delattr()** function will delete the specified attribute from the specified object.
- An error will be shown if the attribute is not found.
- **syntex:-** delattr(object, attribute)

```
class Person:
    name = "John"
    age = 36

delattr(Person, 'name')
print(Person.name)
```

```
class Library:
    def __init__(self, id, name):
        self.bookId = id
        self.bookName = name

book = Library(101, "The Witches")
print(delattr(book, 'bookName'))
print(book.__dict__)    # {'bookId': 101}
```

hasattr()

- The **hasattr()** function returns True if the specified object has the specified attribute, otherwise False.

```
class Person:
    name = "John"
    age = 36
    country = "Norway"

x = hasattr(Person, 'age')

print(x)    # True
```


Object

Ques. What is Object?

- An object is a container that contains data and functionality.
- The object is an entity that has state and behavior. It may be any real-world object like the mouse, keyboard, chair, table, pen, etc.
- Before creating objects, you define a class first. And from the class, you can create one or more objects. The objects of a class are also called **instances** of a class.
- **For example:** if you have an employee class, then it should contain an attribute and method, i.e. an email id, name, age, salary, etc.

```
class Person:
    pass

person = Person()
print(person)
```

```
class car:
    a = "mohit"
    def __init__(self,modelname, year):
        self.modelname = modelname
        self.year = year
    def display(self):
        print(self.modelname,self.year)

c1 = car("Toyota", 2016)
print(c1.a)          # Accessing Object's variables      Output:- mohit
c1.display()         # Accessing Object's functions      Output:- Toyota 2016
```

Modifying Object's properties

```
class Scaler:
    a = 10

# Declaring an object
obj1 = Scaler()
print(obj1.a)    # output:- 10

#Modifying value
obj1.a = 200
print("After modifying the object properties") # output:- After modifying the
object properties
print(obj1.a)    # output:- 200
```

Ques. Delete the Object?

- We can **delete the properties of the object** or object itself by using the **del** keyword.

```
class Student:
    def __init__(self,name):
        self.name = name

# delete the properties of the object
obj = Student("mohit")
print(obj.__dict__)    # Output:- {'name': 'mohit'}
del obj.name
print(obj.__dict__)    # Output:- {}

# Deleting the object itself by del keyword
obj = Student("mohit")
print(obj.__dict__)    # Output:- {'name': 'mohit'}
del obj
print(obj.__dict__)    # Output:- ERROR!
```

Ques. Counting the Number of objects of a Class?

```
class Employee:
    count = 0
    def __init__(self):
        Employee.count = Employee.count + 1

# creating objects
e1 = Employee()
e2 = Employee()
e2 = Employee()
print("The number of Employee:", Employee.count)

Output:- The number of employee: 3
```

Ques. other example:

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def get_avg(self):
        sum = 0
        lgh = len(self.marks)
        for val in self.marks:
            sum = sum+val
        avg = sum/lgh
        print("my name is",self.name,"and my marks is", avg)
# + self.name + " and my marks is " + avg
# creating a new object
stu1 = Student("Mohit", [97,98,96])
stu1.get_avg()

stu1.name= "saxena"
stu1.get_avg()

```

Output:-

```

my name is Mohit and my marks is 97.0
my name is saxena and my marks is 97.0

```

Ques.

```

class Account:
    def __init__(self, acc_no, bal):
        self.acc_no = acc_no
        self.bal = bal

    def debit(self, amount):
        self.bal -= amount
        print("debited amount is ", amount, "my bal is", self.finalAmount())

    def credit(self, amount):
        self.bal += amount
        print("credit amount is ", amount, "my bal is", self.finalAmount())

    def finalAmount(self):
        return self.bal

# creating a new object
stu1 = Account(564654,10000)
stu1.debit(900)      # Output:- debited amount is 900 my bal is 9100
stu1.credit(800)    # Output:- credit amount is 800 my bal is 9900

```

Inheritance

Ques What is Inheritance?

- The process of **inheriting** all the methods and properties of the parent class into a child class is called inheritance.
- **Benefits:-** It provides the reusability of a code. We don't have to write the same code again and again.

Types Of Inheritance?

- **Ques What is Inheritance?**
 - Types Of Inheritance?
 - Single Inheritance:-
 - Multiple Inheritance:-
 - Multi-Level Inheritance:-
 - Hierarchical Inheritance:-
 - Hybrid Inheritance:-

Single Inheritance

- In single inheritance, a child class inherits from a single-parent class. Here is one child class and one parent class.

```
A
|
|
|
B
```

```
# Base class
class Vehicle:
    def Vehicle_info(self):
        print('Inside Vehicle class')

# Child class
class Car(Vehicle):
    def car_info(self):
        print('Inside Car class')

# Create object of Car
car = Car()

# access Vehicle's info using car object
car.Vehicle_info()  # output:- Inside Vehicle class
car.car_info()      # output:- Inside Car class
```

Multiple Inheritance

- In multiple inheritance, a **single child class** is **inherited** from **two or more parent classes**. This means the child class has access to all the methods and attributes of all the parent classes.



```

class Person:
    def person_info(self, name, age):
        print('Inside Person class')
        print('Name:', name, 'Age:', age)

# Parent class 2
class Company:
    def company_info(self, company_name, location):
        print('Inside Company class')
        print('Name:', company_name, 'location:', location)

# Child class
class Employee(Person, Company):
    def Employee_info(self, salary, skill):
        print('Inside Employee class')
        print('Salary:', salary, 'Skill:', skill)

# Create object of Employee
emp = Employee()

# access data
emp.person_info('Jessa', 28)
emp.company_info('Google', 'Atlanta')
emp.Employee_info(12000, 'Machine Learning')
  
```

Output:-

```

Inside Person class
Name: Jessa Age: 28
Inside Company class
Name: Google location: Atlanta
Inside Employee class
Salary: 12000 Skill: Machine Learnings
  
```

Multi-Level Inheritance

- In multilevel inheritance, a class inherits from a child class or derived class. Suppose three classes A, B, C. A is the superclass, B is the child class of A, C is the child class of B. In other words, we can say a chain of classes is called multilevel inheritance.

```
A
|
|
B
|
|
C
```

```
# Base class
class Vehicle:
    def Vehicle_info(self):
        print('Inside Vehicle class')

# Child class
class Car(Vehicle):
    def car_info(self):
        print('Inside Car class')

# Child class
class SportsCar(Car):
    def sports_car_info(self):
        print('Inside SportsCar class')

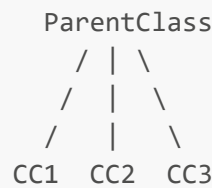
# Create object of SportsCar
s_car = SportsCar()

# access Vehicle's and Car info using SportsCar object
s_car.Vehicle_info()
s_car.car_info()
s_car.sports_car_info()

Output:-
Inside Vehicle class
Inside Car class
Inside SportsCar class
```

Hierarchical Inheritance

- In Hierarchical inheritance, more than one child class is derived from a single parent class. In other words, we can say one parent class and multiple child classes.



```
class Vehicle:
    def info(self):
        print("This is Vehicle")

class Car(Vehicle):
    def car_info(self, name):
        print("Car name is:", name)

class Truck(Vehicle):
    def truck_info(self, name):
        print("Truck name is:", name)

obj1 = Car()
obj1.info()
obj1.car_info('BMW')

obj2 = Truck()
obj2.info()
obj2.truck_info('Ford')
```

Output:-

```
This is Vehicle
Car name is: BMW
This is Vehicle
Truck name is: Ford
```

Hybrid Inheritance

- Hybrid Inheritance is the mixture of two or more different types of inheritance. Here we can have many to many relations between parent classes and child classes with multiple levels.



```

class Vehicle:
    def vehicle_info(self):
        print("Inside Vehicle class")

class Car(Vehicle):
    def car_info(self):
        print("Inside Car class")

class Truck(Vehicle):
    def truck_info(self):
        print("Inside Truck class")

# Sports Car can inherits properties of Vehicle and Car
class SportsCar(Car, Vehicle):
    def sports_car_info(self):
        print("Inside SportsCar class")

# create object
s_car = SportsCar()

s_car.vehicle_info()
s_car.car_info()
s_car.sports_car_info()

```

Output:-

```

Hello Parent1
Hello Child1

```

```

Hello Parent1
Hello Parent2
Hello Child1
Hello Child2

```


Encapsulation

Ques. What is Encapsulation?

- Binding(or wrapping) code and data together into a single unit is known as encapsulation. One object is encapsulated from another object.

```
class BankAccount:
    self.__account_number = account_number # Private attribute (using double
underscore)
    self.__balance = balance # Private attribute

    # Public method to access private balance
    def get_balance(self):
        return self.__balance

    # Public method to modify balance with validation
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            return True
        return False

    # Public method to withdraw with validation
    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return True
        return False

# Creating an instance of the BankAccount
account = BankAccount("123456", 1000)

# Using public methods to interact with private attributes
print(account.get_balance()) # 1000
account.deposit(500)
print(account.get_balance()) # 1500
account.withdraw(200)
print(account.get_balance()) # 1300
```

- **Code of getter & setter in Encapsulation**

```
class Library:
    def __init__(self, id, name):
        self.bookId = id
        self.bookName = name

    def setBookName(self, newBookName): #setters method to set the book name
```

```
self.bookName = newBookName
```

```
def getBookName(self): #getters method to get the book name  
    print(f"The name of book is {self.bookName}")
```

```
book = Library(101,"Hindi")  
book.getBookName()  
book.setBookName("English")  
book.getBookName()
```

Output:-

The name of book is Hindi

The name of book is English

Polymorphism

Ques. What is Polymorphism?

- Polymorphism means the same function name is being used for different types. Each function is differentiated based on its data type and number of arguments. So, each function has a different signature.
- When one task is performed by different ways i.e known as polymorphism.
- A child class inherits all the methods from the parent class.
- we understand that one task can be performed in different ways.
- Polymorphism is ability to use function & method in different ways.

Types of Polymorphism?

- Polymorphism could be static and dynamic both. Overloading is static polymorphism while, overriding is dynamic polymorphism.

1. Duck Typing

2. **Compile time polymorphism (Static) - Method Overloading:-** Overloading is defining functions/methods that have same signatures with different parameters in the same class.

```
class VIP:
    def vsp(self, x=None, y=None):
        if x==None and y==None:
            print("this is python polymorphism")
        elif x!=None and y==None:
            f=1
            for i in range(1, x+1):
                f= f*i
            print(f)
        else:
            print("add:",x+y)
```

```
obj = VIP()
```

```
obj.vsp()
obj.vsp(5)
obj.vsp(5,5)
```

```
output:-
this is python polymorphism
120
add: 10
```

3. Runtime time polymorphism (Dynamic) - Method Overriding:-

1. Overriding is redefining parent class functions/methods in child class with same signature. So, basically the purpose of overriding is to change the behavior of your parent class method.
2. function wo wala call hota jiska hamne obj banya hota hai.

```
class A:
    def vsp(self):
        print("Hiiiiii")
class B(A):
    def vsp(self):
        print("Hello")
```

```
obj = A()
obj.vsp()
```

Output:-
Hiiiiii

4. operator Overloading

```
class A:
    def vsp(self,x):
        self.x = x
    def __add__(self,o):
        return self.x+o.x
```

```
obj1 = A()
obj1.vsp(10)
```

```
obj2 = A()
obj2.vsp(20)
```

```
print(obj1+obj2)
```

Output:- 30

Polymorphism with Class Methods

```
class Monkey:
    def color(self):
        print("The monkey is yellow coloured!")

    def eats(self):
        print("The monkey eats bananas!")

class Rabbit:
    def color(self):
        print("The rabbit is white coloured!")

    def eats(self):
        print("The rabbit eats carrots!")
```

```
mon = Monkey()
rab = Rabbit()
for animal in (mon, rab):
    animal.color()
    animal.eats()
```

Output:-

```
The monkey is yellow coloured!
The monkey eats bananas!
The rabbit is white coloured!
The rabbit eats carrots!
```

Polymorphism with Inheritance

```
class Bird:
    def intro(self):
        print("There are many types of birds.")

    def flight(self):
        print("Most of the birds can fly but some cannot.")

class sparrow(Bird):
    def flight(self):
        print("Sparrows can fly.")

class ostrich(Bird):
    def flight(self):
        print("Ostriches cannot fly.")

obj_bird = Bird()
obj_spr = sparrow()
obj_ost = ostrich()

obj_bird.intro()
obj_bird.flight()

obj_spr.intro()
obj_spr.flight()

obj_ost.intro()
obj_ost.flight()
```

Output:-

```
There are many types of birds.
Most of the birds can fly but some cannot.
There are many types of birds.
Sparrows can fly.
There are many types of birds.
Ostriches cannot fly.
```

constructor

Ques. What is a constructor in Python?

- Constructors are special function/method which is automatically called when an object is created.
- and first parameter is self always.
- In Python, the **init()** method is a special constructor method that is automatically called when an object is created, and it is called once for each instance of the class.
- A constructor is called only once at the time of creation an object.
- if two object are created for a same class, the constructor will be called once for each instance/object.
- jab hum koi **init** function pass nahi karte the class ke ander tab default constructor call hota hai(python ka inbuilt constructor).
- If you don't define an **init()** method in your class, Python automatically provides a default constructor that takes no arguments.

Types of Constructors (Default, Non-parametrized, Parameterized)

- Default Constructor
- Non-parametrized constructor
- Parameterized constructor

Default Constructor

- The default constructor is not present in the source py file.

```
class Employee:  
  
    def display(self):  
        print('Inside Display')
```

```
emp = Employee()  
emp.display()
```

Output:-Inside Display

Default constructor/Non-Parameterized Constructor

- The default constructor is a simple constructor which doesn't accept any arguments. Its definition has only one argument which is a reference to the instance being constructed.

```
class Shot:  
    # Creating default constructor  
    def __init__(self):  
        print("This is a default constructor")  
        self.title = "My name is mohit saxena"
```

```
# a method to display the data member
def display_message(self):
    print(self.title)

# creating object of the class
s1_obj = Shot()           # Output:- This is a default constructor
s1_obj.display_message()  # Output:- My name is mohit saxena
```

parameterized constructor

- constructor with parameters is known as parameterized constructor.
- The parameterized constructor takes its first argument as a reference to the instance being constructed known as self and the rest of the arguments are provided by the programmer.

```
class Triangle:
    # parameterized constructor
    def __init__(self, b, h):
        self.base = b
        self.height = h

    def areaOfTriangle(self):
        self.area = self.base * self.height * 0.5
        print("Area of triangle: " + str(self.area))

# creating object of the class
# this will invoke parameterized constructor
obj = Triangle(5, 14)
print(obj.base)           # 5
print(obj.height)         # 14
obj.areaOfTriangle()      # Area of triangle: 35.0
```

- Example2:-

```
class Student:
    # constructor
    # initialize instance variable
    def __init__(self, name):
        print('Inside Constructor')
        self.name = name
        print('All variables initialized')

    # instance Method
    def show(self):
        print('Hello, my name is', self.name)

# create object using constructor
s1 = Student('Mohit')
s1.show()
```

```
Output:-  
Inside Constructor  
All variables initialized  
Hello, my name is Mohit
```


Ques. Constructor With Default Values?

- Python allows us to define a constructor with default values. The default value will be used if we do not pass arguments to the constructor at the time of object creation.

```
class Student:
    # constructor with default values age and classroom
    def __init__(self, name, age=12, classroom=7):
        self.name = name
        self.age = age
        self.classroom = classroom

    # display Student
    def show(self):
        print(self.name, self.age, self.classroom)

# creating object of the Student class
emma = Student('Emma')
emma.show()

kelly = Student('Kelly', 13)    # Output:- Emma 12 7
kelly.show()                  # Output:- Kelly 13 7
```

Ques. Constructor Overloading?

- Python does not support constructor overloading.
- If we define multiple constructors then, the interpreter will consider only the last constructor and throws an error if the sequence of the arguments doesn't match as per the last constructor.

```
class Student:
    # one argument constructor
    def __init__(self, name):
        print("One arguments constructor")
        self.name = name

    # two argument constructor
    def __init__(self, name, age):
        print("Two arguments constructor")
        self.name = name
        self.age = age

# creating first object
emma = Student('Emma')

# creating Second object
kelly = Student('Kelly', 13)

Output:- TypeError: __init__() missing 1 required positional argument: 'age'
```

Destructors

Ques. What is Destructors?

- Destructor is a special method that is called when an **object** is **deleted** or **destroyed**.
- The special method `__del__()` is used to define a destructor. For example, when we execute `del object_name` destructor gets called automatically and the object gets garbage collected.

```
class Student:

    # constructor
    def __init__(self, name):
        self.name = name
        print('Object is created')

    def show(self):
        print('Hello, my name is', self.name)

    # destructor
    def __del__(self):
        print('Object is being destroyed')

# create object
s1 = Student('Emma')    # call init method automatically
s1.show()

# delete object
del s1                  # # call del method automatically
```

Output:-

Object is created

Hello, my name is Emma

Object is being destroyed

Python Access Modifiers

Python Access Modifiers

- **Public Member:** Accessible anywhere from outside the class.
- **Private Member:** private attributes and method Accessible only within the class.
- **Protected Member:** Accessible within the class and it's sub-classes.

```
#defining class Student
class Student:
    #constructor is defined
    def __init__(self, name, age, salary):
        self.age = age           # public Attribute
        self._name = name       # protected Attribute
        self.__salary = salary  # private Attribute

    def _funName(self):          # protected method
        pass

    def __funName(self):         # private method
        pass

# object creation
obj = Student('Mohit', 53434)
```

Public access modifier:-

- By default, all the variables and member functions of a class are public in a python program.

```
class MyClass:
    def __init__(self, name, age):
        self.name = name  # Public attribute
        self.age = age    # Public attribute

    def greet(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

# Create an instance of the class
my_object = MyClass("Mohit", 30)

# Access public attributes and call public methods
print(my_object.name)  # Output: Mohit
print(my_object.age)   # Output: 30
my_object.greet()      # Output: Hello, my name is Mohit and I am 30 years old.
```

protected access modifier:-

- **protected Access Modifier:-** Accessible within the class and it's sub-classes.
- adding a prefix `_`(single underscore) to a variable name makes it protected.

```
#example1
class MyClass:
    def __init__(self, name):
        self._name = name # Protected attribute

    def _protected_method(self): # Protected method
        print(f"Hello, {self._name}!")

    def public_method(self):
        self._protected_method()

class SubClass(MyClass):
    def __init__(self, name, age):
        super().__init__(name)
        self.age = age

    def access_protected(self):
        print(f"My name is {self._name} and I am {self.age} years old.")

objsubclass = SubClass("mohit", 30)
# access the Protected variable
print(objsubclass._name) # Output:- Mohit
# access the prProtected method
objsubclass._protected_method() # Output:- Hello, mohit!
objsubclass.access_protected() # Output:- My name is mohit and I am 30 years old.
```

private access modifier:-

- Private attributes are denoted with a double underscore prefix (__) and are intended for internal use within a class, not to be accessed or modified directly from outside.
- Private attributes & method are meant to be used only within the class and are not accessible from the outside the class.
- agar humne double underscore se kisi ko private kar diya hai to use usi class ke function call kar payenge.
- hum directly hello function ko call nahi kar sakte hai kyuki hello function private hai agar hame karana hai to usi class mai dusra function banakar welcome banakar usme call karenge phir welcome function ko call karenge to call ho jayega.

```
class MyClass:
    __name = "mohit saxena"
    def __init__(self, public_attr, private_attr):
        self.public_attr = public_attr # Public attribute
        self.__private_attr = private_attr # Private attribute

    def public_method(self):
        print(f"Public attribute: {self.public_attr}")
        print(f"Private attribute: {self.__private_attr}") # Accessing private
attribute within the class

    def __private_method(self):
        print("Private method call")

    def get_private_method_result(self):
        # Call the private method from within a public method
        return self.__private_method()

    def get_name(self):
        return MyClass.__name

# ++++++ Create an instance ++++++
my_object = MyClass("public value", "private value")

# ++++++ access the variable ++++++
# access the class variable using mangled
print(MyClass.__name) # Output:- Mohit saxena
print(my_object.__name) # Output:- Mohit saxena

# access the class variable using get method
print(my_object.get_name())

# +++ Accessing public attributes and methods +++++
print(my_object.public_attr) # Output:- public value
my_object.public_method() # Output:- Public attribute: public value , Private
attribute: private value
```

```
# ++++++ Accessing the private method using get method ++++++
my_object.get_private_method_result() # Output:-
my_object.get_private_method_result()

# Accessing the private method using its mangled name
my_object._MyClass__private_method() # Private method call

# ++++++ Attempting to access a private attribute directly will raise an
AttributeError ++++++
# print(my_object.__private_attr) #This will raise an error
```

```
class Account:
    def __init__(self, acc_no, acc_pass):
        self.acc_no = acc_no
        self.__acc_pass = acc_pass

    def reset_pass(self):
        print(self.__acc_pass)

acc1 = Account("2582123", "pass@123")
print(acc1.reset_pass()) # Output:- pass@123
```

using mangled name

```
class MyClass:
    def __init__(self):
        self.name = "mohit"
        self.__private_method()

    def __private_method(self):
        print("This is a private method.")

# Create an instance of MyClass
obj = MyClass()

# Attempting to call the private method directly will raise an AttributeError
# obj.__private_method() # Uncommenting this line will raise an error

# Accessing the private method using its mangled name
"""
In Python, a mangled name refers to the modified name of a class attribute or
method that has been defined with a double underscore prefix (__). This name
mangling is a mechanism used to prevent name clashes in subclasses and to provide
a level of encapsulation.
```

When you define a method or attribute with a double underscore, Python automatically changes its name to include the class name as a prefix. This is done to avoid accidental access or modification of the method or attribute from outside

```
the class or from subclasses.  
"""  
obj._MyClass__private_method() # This will work
```

Why are access modifiers important in programming?

- Access modifiers improve security by limiting the accessibility of elements and preventing unauthorized access.
- The access modifier is a keyword that controls the accessibility of the class, hides the internal implementations, and enhances security.
- Access modifiers are important as they control the accessibility of code, promote encapsulation, hide the internal implementation, and maintain security.

Ques. What is the super() Function?

- Super() method is used to access method and properties of the parent class.
- The **super** function returns a temporary object of the parent class that allows us to call a parent class method inside a child class method.
- **Benefits** of using the super() function.
 - We are not required to remember or specify the parent class name to access its methods.
 - We can use the super() function in both single and multiple inheritances.
 - The super() function supports code reusability as there is no need to write the entire function

```
class Company:
    def company_name(self):
        return 'Google'

class Employee(Company):
    def info(self):
        # Calling the superclass method using super()function
        c_name = super().company_name()
        print("Jessa works at", c_name)

# Creating object of child class
emp = Employee()
emp.info()
```

Output:- Jessa works at Google

super() with Single Inheritance

```
class Animal:
    def smell(self, name):
        print(name, "can smell")

class Dog(Animal):
    def __init__(self):
        super().smell("Dog")

    def bark(self):
        print("Dog can bark")

dog = Dog()      # Output:- Dog can smell
dog.bark()      # Output:- Dog can bark
```

```
class Parent:
    def __init__(self, name):
        self.name = name

    def display(self):
```



```

        print(f"Parent name: {self.name}")

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Call the constructor of the parent class
        self.age = age

    def display(self):
        super().display()      # Call the display method of the parent class
using super keyword
        Parent.display(self)  # Call the display method of the parent class
using class name
        print(f"Child age: {self.age}")

# Example usage
parent = Parent("John")
parent.display()    # output:- Parent name: John

child = Child("Alice", 10)
child.display()    # output:- Parent name: Alice, Parent name: Alice, Child age: 10

```

super() with Multiple Inheritance

```

class Parent:
    def __init__(self):
        print("This is the parent class")

class Parent1:
    def __init__(self):
        print("This is the parent1 class")

class Child(Parent1, Parent):
    def __init__(self):
        ##Calling constructor of the Parnet 1 class
        super().__init__()

ob = Child() # output:- This is the parent1 class

# if we have inherit class parent and parent1
class Child(Parent, Parent1):
    def __init__(self):
        ##Calling constructor of the Parnet 1 class
        super().__init__()

ob = Child() # output:- This is the parent class

```

Ques. What is MRO(Method Resolution Order) / Diamond Problem?

- MRO is a concept used in **inheritance**.
- MRO stands for Method Resolution Order. MRO defines the order of the inherited methods in the child class.
- In Python, the MRO is from **bottom to top** and **left to right**. This means that, first, the method is searched in the class of the object. If it's not found, it is searched in the immediate super class. In the case of multiple super classes, it is searched left to right, in the order by which was declared by the developer.

```
# Example1
A
|
B
class A:
    def method(self):
        print("A.method() called")

class B(A):
    def method(self):
        print("B.method() called")

b = B()
b.method() # output:- B.method() called
```

```
# Example2
B  A
|  |
|__|
|_C_|
class A:
    def method(self):
        print("A.method() called")

class B:
    pass

class C(B, A):
    pass

c = C()
c.method() # Outpur:- print("A.method() called")

# example_3
class A:
    def method(self):
        print("A.method() called")

class B:
    def method(self):
```

```

    print("B.method() called")

class C(A, B):
    pass

class D(C, B):
    pass

d = D()
d.method()  # Output:- A.method() called

```

```

# Example:-
class father():
    def display(self):
        print("father class method")

class mother():
    def display(self):
        print("mother class method")

class son(mother, father):                # left to right
    def showS(self):
        print("son class method")

obj = son()
obj.showS()    # Output:- son class method
obj.display()  # Output:- mother class method

# if we change the ordering then
class son(father, mother):                # left to right
    def showS(self):
        print("son class method")

obj = son()
obj.showS()    # Output:- son class method
obj.display()  # Output:- father class method

-----

# Using Constructor
class father():
    def __init__(self):
        super().__init__()    # Calling Parent Class Constructor
        print("father class Constructor")
    def showF(self):
        print("father class method")

class mother():
    def __init__(self):
        super().__init__()    # Calling Parent Class Constructor
        print("mother class Constructor")

```

```

    def showM(self):
        print("mother class method")
class son(father, mother):
    # left to right
    def __init__(self):
        super().__init__() # Calling Parent Class Constructor 1st Wala
        print("son class Constructor")
    def showS(self):
        print("son class method")

obj = son()

```

Output:-

```

mother class Constructor
father class Constructor
son class Constructor

```

```

# call parent call using obj
class mother():
    def __init__(self):
        super().__init__() # Calling Parent Class Constructor
        print("mother class Constructor")

class father():
    def __init__(self):
        print("father class Constructor")

class son(father, mother):
    # left to right
    def __init__(self):
        print("son class Constructor")

obj = son()
obj.__class__.__bases__[1].__init__(obj) # 0 if father or 1 is mother class
son(father, mother):

#output:
son class Constructor
mother class Constructor

```

Ques. What is Abstract Class?

- We cannot create an abstract class in Python directly. However, Python does provide a module that allows us to define abstract classes. The module we can use to create an abstract class in Python is `abc`(abstract base class) module. **Rule**
- we can not create objects of an abstract class (abstract class ka hum object nahi bna sakte hai).
- It is not necessary to declare all methods abstract in a abstract class.
- Abstract class can have abstract method and concrete method.
- If there is any abstract method in a class, that class must be abstract.
- The abstract methods of an abstract class must be defined in its child class/subclass.

Ques. When use abstract class?

- We use abstract class when there are some common feature shared by all the objects as they are.

How to Create an Abstract Method

- Import the required tools from the `abc` module.
- Use the `@abstractmethod` decorator above your method.
- Make the class inherit from `ABC` (Abstract Base Class).

```
from abc import ABC, abstractmethod # Step 1

class Animal(ABC): # Step 2: Inherit from ABC

    @abstractmethod # Step 3: Use decorator
    def make_sound(self): # Abstract method
        pass
```

Example:-

```
# gun is common features
# Defence Force
# Gun:- Ak 47
# Area:- --
-----
# army:- Gun Ak 47, Area:- Land
# Air Force:- Gun Ak 47, Area:- Sky
# Navy:- Gun Ak 47, Area:- Sea
```

```
from abc import ABC
class <Abstract_Class_Name>(ABC):
```

```
from abc import ABC, abstractmethod
class Father(ABC):
    @abstractmethod
    def disp(self,a,b):
        pass

    def show(self):
        print("concreate class")

class child(Father):
    def disp(self,a,b):
        print(a+b)
        print("defining abstrat class")

class child2(Father):
    def disp(self,a,b):
        print(a*b)
        print("defining abstrat class")

c = child()
c.disp(10,50)
c.show()

c2 = child2()
c2.disp(10,30)
```

👉 Output:-

```
60
defining abstrat class
concreate class
300
defining abstrat class
```

Built-In Class Attributes?

- The built-in class attributes provide us with information about the class.
- Using the dot (.) operator, we may access the built-in attributes.
- The built-in class attributes in python are listed below –

1. **dict** :-

- The **dict** attribute is a dictionary containing the namespace of a class or instance.
- It holds the attributes and methods defined for the class or object, allowing dynamic inspection and manipulation of its members.
- Example:-

```
class Person:
    'My name is mohit saxena'

    def __init__(self):
        print("The __init__ method")

print(Person.__dict__)
```

Output:-

```
{'__module__': '__main__', '__init__': <function Person.__init__ at
0x7c736ba3c7c0>, '__dict__': <attribute '__dict__' of 'Person' objects>,
'__weakref__': <attribute '__weakref__' of 'Person' objects>, '__doc__': None}
```

2. **doc**

- The **doc** attribute contains the string of the class documents.

```
class Person:
    'My name is mohit saxena'

    def __init__(self):
        print("The __init__ method")

print(Person.__doc__)
```

Output:-

```
My name is mohit saxena
```

3. **name**

- The **name** attribute provides the name of the class to which it belongs.
- It is used to retrieve the name of a class within the class definition or in instances of the class.

```
class Person:
    'My name is mohit saxena'

    def __init__(self):
```

```
print("The __init__ method")

print(Person.__name__)
```

Output:-
Person

4. module

- The **module** attribute in Python represents the name of the module in which a class is defined.
- It allows you to access and retrieve the module name to which a particular class belongs. **module** contains the name of that module.
- This attribute is especially useful when working with classes defined in modules.

```
class Person:
    'My name is mohit saxena'

    def __init__(self):
        print("The __init__ method")

print(Person.__module__)
```

Output:-
__main__

5. bases

- The **bases** attribute in Python is used to access a tuple containing the base classes of a class.
- It provides information about the direct parent classes from which the class inherits.

```
class Person:
    'My name is mohit saxena'

    def __init__(self):
        print("The __init__ method")

print(Person.__bases__)
```

Output:-
(<class 'object'>,)

Ques. What is Method Overloading?

- Two or more methods have the same name but different numbers of parameters, These methods are called overloaded methods.

```
class Addition:
    # first sum for 2 params
    def my_sum(self, a, b):
        return a + b

    # second overloaded sum for 3 params
    def my_sum(self, a, b, c):
        return a + b + c

obj = Addition()
# print(obj.my_sum(3, 4))
print(obj.my_sum(3, 4, 5))
```

Output: 12

Ques. What is Method Overriding?

- When a child class method has the same name, same parameters, and same return type as a method.

```
class Vehicle:
    def max_speed(self):
        print("max speed is 100 Km/Hour")

class Car(Vehicle):
    # overridden the implementation of Vehicle class
    def max_speed(self):
        print("max speed is 200 Km/Hour")

# Creating object of Car class
car = Car()
car.max_speed()
```

Output:- max speed is 200 Km/Hour

Ques. What is Method?

- The method is a function that is associated with an object. In Python, a method is not unique to class instances. Any object type can have methods.

Ques. Types Of Methods In Python?

There are three types of methods in Python.

1. Instance Methods.
2. Class Methods.
3. Static Methods.

- **Instance Method**

- when we create classes in python. If we want to print an instance variable or instance method we must create an object of that required class.
- If we are using self as a function parameter or in front of a variable, that is nothing but the calling instance itself.
- As we are working with instance variables we use self keyword.
- **Note:-** Instance variables are used with instance methods.

```
class Student:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def avg(self):
        return (self.a + self.b) / 2

s1 = Student(10, 20)
print( s1.avg() )
```

Output:- 15.0

- **Class Method**

- classmethod() function returns a class method as output for the given function.
- If we want to create a class method we must use **@classmethod** decorator and **cls** as a parameter for that function.

```
class Student:
    name = 'Student'
    def __init__(self, a, b):
        self.a = a
        self.b = b

    @classmethod
    def info(cls):
        return cls.name
```

```
print(Student.info())
```

Output:- Student

- **Static Method**

- A static method can be called without an object for that class, using the class name directly. If you want to do something extra with a class we use static methods.
- A static method in python must be created by decorating it with **@staticmethod**

```
class Student:
    name = 'Student'
    def __init__(self, a, b):
        self.a = a
        self.b = b

    @staticmethod
    def info():
        return "This is a student class"
```

```
print(Student.info())
```

Output:- This a student class

```
class MethodTypes:

    name = "Ragnar"

    def instanceMethod(self):
        # Creates an instance attribute through keyword self
        self.lastname = "Lothbrock"
        print(self.name)
        print(self.lastname)

    @classmethod
    def classMethod(cls):
        # Access a class attribute through keyword cls
        cls.name = "Lagertha"
        print(cls.name)

    @staticmethod
    def staticMethod():
        print("This is a static method")

# Creates an instance of the class
m = MethodTypes()
# Calls instance method
m.instanceMethod()
```

```
MethodTypes.classMethod()  
MethodTypes.staticMethod()
```

Output:-

Ragnar

Lothbrock

Lagertha

This **is** a static method

What is static method?

- static method don't use the self parameter.
- static method ko hum object ke sath bhi call kar sakte hai or class ke stah bhi.

```
class Student:

    @staticmethod
    def hello():
        print("hello")

stu1 = Student()
stu1.hello()      # output:- Hello
Student.hello()   # output:- Hello
```